
BÁO CÁO BÀI TẬP VỀ NHÀ 5

GIẢI HỆ PHƯƠNG TRÌNH TUYẾN TÍNH BẰNG PHƯƠNG PHÁP JACOBI VÀ GAUSS-SEIDEL

Họ và tên: Nguyễn Trần Gia Bảo

MSSV: 23137003

Môn học: Vật lý tính toán (PHY10532)

Giảng viên: TS. Vũ Quang Tuyên

Ngày 8 tháng 4 năm 2026

Mục lục

1	Giới thiệu và nội dung bài tập	2
2	Cơ sở lý thuyết	2
2.1	Bài toán tổng quát	2
2.2	Phương pháp Jacobi	2
2.3	Phương pháp Gauss–Seidel	3
2.4	Điều kiện hội tụ	3
2.5	Tiêu chuẩn dừng	3
2.6	Kỹ thuật phương trình chuẩn tắc (Normal Equations)	3
2.6.1	Chứng minh	3
2.6.2	Tóm tắt ưu và nhược điểm	4
3	Phân tích chương trình	4
3.1	Hàm Jacobi	4
3.2	Hàm Gauss–Seidel	5
3.3	Hàm ghi file kết quả	6
3.4	Hàm tính toán tổng hợp	7
4	Kết quả và thảo luận	7
4.1	Bài 1: Hệ phương trình 4×4	7
4.1.1	Đề bài	7
4.1.2	Kiểm tra điều kiện hội tụ	7
4.1.3	Kết quả	8
4.2	Bài 2: Bài toán cột kéo (Truss)	8
4.2.1	Sơ đồ hệ thanh	8
4.2.2	Thiết lập hệ phương trình	8
4.2.3	Khó khăn 1: Phần tử đường chéo bằng 0	9
4.2.4	Khó khăn 2: Không hội tụ	10
4.2.5	Giải pháp: Phương trình chuẩn tắc $A^T Ax = A^T B$	10
4.2.6	Minh họa bằng ví dụ nhỏ	11
4.2.7	Kết quả bài toán cột kéo	11
4.2.8	Biện luận kết quả sau khi giải	11
5	Kết luận	12

1. Giới thiệu và nội dung bài tập

Bài tập yêu cầu giải hệ phương trình tuyến tính $Ax = B$ bằng hai phương pháp lặp: **Jacobi** và **Gauss–Seidel**. Cụ thể gồm hai phần:

1. **Bài 1:** Giải hệ 4×4 cho trước bằng cả hai phương pháp, so sánh tốc độ hội tụ.
2. **Bài 2 (Bài toán cột kéo):** Thiết lập và giải hệ 8×8 xuất phát từ bài toán cân bằng lực tại các nút của hệ thanh chịu lực.

2. Cơ sở lý thuyết

2.1. Bài toán tổng quát

Xét hệ n phương trình tuyến tính với n ẩn:

$$Ax = B, \quad (1)$$

trong đó $A \in \mathbb{R}^{n \times n}$ là ma trận hệ số, $x \in \mathbb{R}^n$ là vector ẩn và $B \in \mathbb{R}^n$ là vector vế phải.

Để giải bằng phương pháp lặp, ta tách ma trận A thành

$$A = D + L + U,$$

với D là ma trận đường chéo, L là phần tam giác dưới (không chứa đường chéo) và U là phần tam giác trên. Điều kiện tiên quyết là mọi phần tử đường chéo phải khác không: $a_{ii} \neq 0$ với mọi i .

2.2. Phương pháp Jacobi

Từ phương trình thứ i của hệ Phương trình (1):

$$\sum_{j=1}^n a_{ij} x_j = b_i,$$

ta tách riêng x_i :

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j \right). \quad (2)$$

Trong phương pháp Jacobi, ở mỗi bước lặp thứ $(k+1)$, ta dùng *toàn bộ* giá trị cũ $x^{(k)}$ để tính giá trị mới:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{\substack{j=1 \\ j \neq i}}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n. \quad (3)$$

Dưới dạng ma trận, công thức lặp Jacobi có thể viết:

$$x^{(k+1)} = D^{-1}(B - (L + U)x^{(k)}) = -D^{-1}(L + U)x^{(k)} + D^{-1}B.$$

Ma trận lặp tương ứng là $T_J = -D^{-1}(L + U)$.

2.3. Phương pháp Gauss–Seidel

Ý tưởng cải tiến: khi tính $x_i^{(k+1)}$, các thành phần $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ đã được tính xong ở bước lặp hiện tại, nên ta dùng chúng thay vì giữ giá trị cũ:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n. \quad (4)$$

Dưới dạng ma trận:

$$x^{(k+1)} = -(D + L)^{-1} U x^{(k)} + (D + L)^{-1} B.$$

Ma trận lặp tương ứng là $T_{GS} = -(D + L)^{-1} U$.

2.4. Điều kiện hội tụ

Cả hai phương pháp hội tụ khi bán kính phổ (spectral radius) của ma trận lặp nhỏ hơn 1. Một điều kiện đủ phổ biến là **ma trận chéo trội nghiêm ngặt theo hàng**:

$$|a_{ii}| > \sum_{\substack{j=1 \\ j \neq i}}^n |a_{ij}|, \quad \forall i = 1, \dots, n. \quad (5)$$

Khi Phương trình (5) được thỏa, cả Jacobi lẫn Gauss–Seidel đều hội tụ. Trong thực tế, Gauss–Seidel thường hội tụ nhanh hơn Jacobi vì sử dụng thông tin mới nhất ngay khi có.

2.5. Tiêu chuẩn dừng

Quá trình lặp dừng khi:

$$\text{err}^{(k)} = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon, \quad (6)$$

với ε là ngưỡng sai số cho trước. Ngoài ra, ta giới hạn số lần lặp tối đa $N_{\max}$ để tránh vòng lặp vô hạn khi phương pháp không hội tụ.

2.6. Kỹ thuật phương trình chuẩn tắc (Normal Equations)

Trong bài toán cột kéo, ma trận A không chéo trội và có nhiều phần tử đường chéo bằng 0, nên Jacobi/Gauss–Seidel đều *không hội tụ* khi áp dụng trực tiếp. Để khắc phục, ta nhân cả hai vế của Phương trình (1) với A^T :

$$A^T A x = A^T B. \quad (7)$$

2.6.1. Chứng minh

1. **Tương đương nghiệm:** Nếu x^* là nghiệm của $Ax = B$ thì hiển nhiên $A^T A x^* = A^T B$. Ngược lại, nếu A khả nghịch ($\det A \neq 0$) thì $A^T A$ cũng khả nghịch (vì $\det(A^T A) = (\det A)^2 \neq 0$), do đó hệ Phương trình (7) có nghiệm duy nhất trùng với nghiệm hệ gốc.

2. Ma trận $A^T A$ đối xứng:

$$M = A^T A \implies M^T = (A^T A)^T = A^T (A^T)^T = A^T A = M.$$

3. Ma trận $A^T A$ xác định dương: Với mọi $v \neq 0$:

$$v^T M v = v^T (A^T A) v = (Av)^T (Av) = \|Av\|^2.$$

Nếu A khả nghịch thì $Av \neq 0$ khi $v \neq 0$, do đó $\|Av\|^2 > 0$, tức M xác định dương.

4. Đường chéo luôn dương: Phần tử đường chéo của $M = A^T A$:

$$m_{ii} = \sum_{k=1}^n a_{ki}^2 > 0,$$

vì cột thứ i của A có ít nhất một phần tử khác 0. Do đó không có đường chéo nào bằng 0, giải quyết hoàn toàn vấn đề chia cho 0.

5. Gauss–Seidel luôn hội tụ: Theo định lý Ostrowski–Reich, phương pháp Gauss–Seidel hội tụ cho mọi ma trận đối xứng xác định dương. Vì $A^T A$ thỏa mãn cả hai tính chất này, Gauss–Seidel được đảm bảo hội tụ.

2.6.2. Tóm tắt ưu và nhược điểm

Ưu điểm:

- Đảm bảo đường chéo dương (không chia cho 0).
- Ma trận hệ số mới đối xứng, xác định dương.
- Gauss–Seidel được đảm bảo hội tụ theo lý thuyết.

Nhược điểm: Số điều kiện tăng bình phương:

$$\kappa(A^T A) = \kappa(A)^2,$$

nên tốc độ hội tụ có thể chậm hơn so với các phương pháp trực tiếp.

3. Phân tích chương trình

Chương trình được viết bằng Python trong notebook BTVN5.ipynb.

3.1. Hàm Jacobi

```
1 def Jacobi(N_max, epsilon, A, B):
2     n = len(B)
3     x_old = n * [0]
4     x_new = n * [0]
5     err_mang = n * [100]
6     err = np.max(err_mang)
```

```

7     x_matrix = []
8     err_matrix = []
9     x_matrix.append(x_old[:])
10    for q in range(N_max):
11        if err > epsilon:
12            for i in range(n):
13                tong = 0
14                for j in range(n):
15                    if i != j:
16                        tong = tong + A[i][j] * x_old[j]
17                x_new[i] = 1 / A[i][i] * (-tong + B[i])
18                err_mang[i] = np.abs(x_new[i] - x_old[i])
19            err = np.max(err_mang)
20            x_matrix.append(x_new[:])
21            err_matrix.append(err)
22            x_old[:] = x_new[:]
23        else:
24            break
25    return q, x_matrix, err_matrix

```

Giải thích chi tiết:

- **Dòng 2–5:** Khởi tạo vector nghiệm cũ và mới bằng 0; mảng sai số ban đầu bằng 100 để đảm bảo vòng lặp chạy ít nhất một lần.
- **Dòng 10–18:** Vòng lặp chính. Với mỗi ẩn x_i , tính tổng $\sum_{j \neq i} a_{ij} x_j^{(k)}$ dùng hoàn toàn giá trị cũ x_{old} rồi cập nhật theo Phương trình (3). Đây là đặc trưng Jacobi: *toàn bộ* x_{old} giữ nguyên trong suốt vòng lặp trong.
- **Dòng 19–22:** Tính sai số max-norm, lưu lịch sử nghiệm và sai số.
- **Dòng 23:** Cập nhật $x_{\text{old}} \leftarrow x_{\text{new}}$ cho bước tiếp.

3.2. Hàm Gauss–Seidel

```

1  def Gauss_Seidel(N_max, epsilon, A, B):
2      n = len(B)
3      x_old = n * [0]
4      x_new = n * [0]
5      err_mang = n * [100]
6      err = np.max(err_mang)
7      x_matrix = []
8      err_matrix = []
9      x_matrix.append(x_old[:])
10     for q in range(N_max):
11         if err > epsilon:
12             for i in range(n):
13                 tong1 = tong2 = 0
14                 for j in range(i):
15                     tong1 = tong1 + A[i][j] * x_new[j]
16                 for j in range(i+1, n):
17                     tong2 = tong2 + A[i][j] * x_old[j]

```

```

18         x_new[i] = 1/A[i][i] * (-tong1 - tong2 + B[i])
19         err_mang[i] = np.abs(x_new[i] - x_old[i])
20         err = np.max(err_mang[:])
21         x_matrix.append(x_new[:])
22         err_matrix.append(err)
23         x_old[:] = x_new[:]
24     else:
25         break
26     return q, x_matrix, err_matrix

```

Điểm khác biệt quan trọng so với Jacobi nằm ở dòng 13–18:

- tong1 (dòng 14–15): Tính $\sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)}$ – dùng giá trị *mới* $x_{\text{new}}[j]$ cho các ẩn $j < i$ đã cập nhật trong bước lặp hiện tại.
- tong2 (dòng 16–17): Tính $\sum_{j=i+1}^n a_{ij} x_j^{(k)}$ – vẫn dùng giá trị *cũ* $x_{\text{old}}[j]$ cho các ẩn $j > i$ chưa cập nhật.
- Chính nhờ việc tận dụng ngay giá trị mới mà Gauss-Seidel thường hội tụ nhanh hơn Jacobi.

3.3. Hàm ghi file kết quả

```

1 def luu_file(q, x_matrix, err_matrix, filename):
2     so_nghiem = len(x_matrix[0])
3     heading = ""
4     for i in range(so_nghiem):
5         label = f"x_{i+1}"
6         heading = heading + f"{label:>15s} "
7     with open(f"{filename}", "w", encoding="utf-8") as file:
8         file.write(f"Ket qua tinh toan sau {q} lan lap:\n\n")
9         file.write(f"#" * 100 + "\n\n")
10        file.write(f"{'i':>5s}" + heading + f"{'sai_so':>20s}\n")
11        for i in range(q):
12            hang = f"{i+1:>5d}"
13            for x in x_matrix[i]:
14                hang = hang + f"{x:>15.9f} "
15            hang += f"{err_matrix[i]:>20.6e}\n"
16            file.write(hang)

```

Hàm này ghi toàn bộ lịch sử lặp (nghiệm ở mỗi bước và sai số tương ứng) ra file `.txt`, giúp theo dõi chi tiết quá trình hội tụ.

3.4. Hàm tính toán tổng hợp

```

1 def tinhtoan(N_max, epsilon, A, B, filename):
2     q_jacobi, x_matrix_jacobi, err_matrix_jacobi = \
3         Jacobi(N_max, epsilon, A, B)
4     q_gauss_seidel, x_matrix_gauss_seidel, err_matrix_gauss_seidel = \
5         Gauss_Seidel(N_max, epsilon, A, B)
6     filename_jacobi = f"{filename}_jacobi.txt"
7     filename_gauss_seidel = f"{filename}_gauss_seidel.txt"
8     luu_file(q_jacobi, x_matrix_jacobi,
9             err_matrix_jacobi, filename_jacobi)
10    luu_file(q_gauss_seidel, x_matrix_gauss_seidel,
11            err_matrix_gauss_seidel, filename_gauss_seidel)

```

Hàm `tinhtoan` đóng vai trò điều phối: gọi lần lượt Jacobi và Gauss-Seidel trên cùng hệ phương trình, rồi lưu kết quả vào hai file riêng biệt để tiện so sánh.

4. Kết quả và thảo luận

4.1. Bài 1: Hệ phương trình 4×4

4.1.1. Đề bài

Giải hệ phương trình:

$$\begin{bmatrix} 10 & -1 & 2 & 0 \\ -1 & 11 & -1 & 3 \\ 2 & -1 & 10 & -1 \\ 0 & 3 & -1 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 6 \\ 25 \\ -11 \\ 15 \end{bmatrix}, \quad (8)$$

với $\varepsilon = 10^{-9}$ và $N_{\max} = 1000$.

```

1 N_max = 1000
2 A = [[10, -1, 2, 0],
3      [-1, 11, -1, 3],
4      [2, -1, 10, -1],
5      [0, 3, -1, 8]]
6 B = [6, 25, -11, 15]
7 epsilon = 1e-9
8 tinhtoan(N_max, epsilon, A, B, "baitap1")

```

4.1.2. Kiểm tra điều kiện hội tụ

Xét tính chéo trội theo hàng:

$$\begin{aligned} |a_{11}| &= 10 > |-1| + |2| + |0| = 3, \\ |a_{22}| &= 11 > |-1| + |-1| + |3| = 5, \\ |a_{33}| &= 10 > |2| + |-1| + |-1| = 4, \\ |a_{44}| &= 8 > |0| + |3| + |-1| = 4. \end{aligned}$$

Mà trận A là chéo trội nghiêm ngặt $\Rightarrow$ cả Jacobi lẫn Gauss-Seidel đều đảm bảo hội tụ.

4.1.3. Kết quả

Nghiệm chính xác của hệ Phương trình (8) (kiểm chứng bằng `np.linalg.solve`) là:

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = -1, \quad x_4 = 1.$$

Bảng 1: So sánh tốc độ hội tụ của Jacobi và Gauss–Seidel cho Bài 1 ($\varepsilon = 10^{-9}$).

	Jacobi	Gauss–Seidel
Số lần lặp	26	11
Nghiệm x_1	1.000000000	1.000000000
Nghiệm x_2	2.000000000	2.000000000
Nghiệm x_3	-1.000000000	-1.000000000
Nghiệm x_4	1.000000000	1.000000000

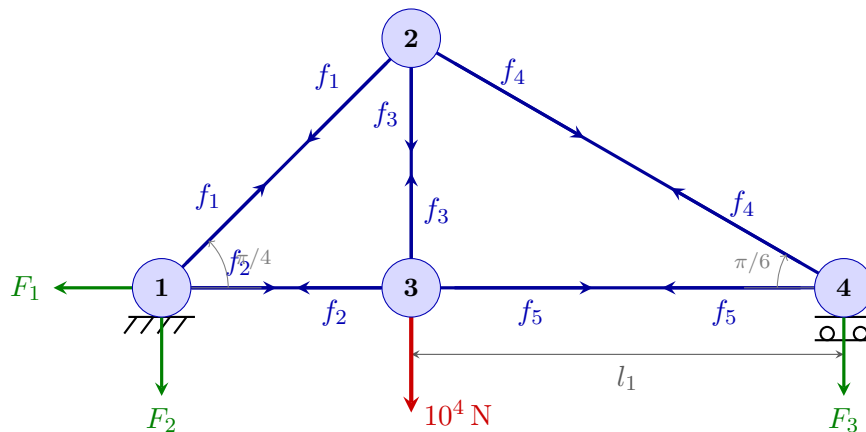
Nhận xét:

- Cả hai phương pháp đều hội tụ về nghiệm đúng.
- Gauss–Seidel hội tụ nhanh hơn Jacobi (ít bước lặp hơn), phù hợp với lý thuyết.
- Ma trận chéo trội mạnh giúp cả hai phương pháp hội tụ nhanh.

4.2. Bài 2: Bài toán cột kéo (Truss)

4.2.1. Sơ đồ hệ thanh

Hình 1 minh họa hình học của hệ thanh, các phản lực liên kết và tải trọng ngoài tác dụng lên kết cấu.



Hình 1: Sơ đồ hệ thanh của bài toán cột kéo, gồm bốn nút, năm thanh, ba phản lực liên kết và tải trọng tập trung 10^4 N tại nút 3.

4.2.2. Thiết lập hệ phương trình

Từ điều kiện cân bằng lực tại các nút của hệ thanh trên Hình 1, ta thu được hệ 8×8 với các ẩn F_1, F_2, F_3 (phản lực) và f_1, f_2, f_3, f_4, f_5 (nội lực các thanh):

```

1 A_cotkeo = [
2     [-1, 0, 0, np.sqrt(2)/2, 1, 0, 0, 0],
3     [0, -1, 0, np.sqrt(2)/2, 0, 0, 0, 0],
4     [0, 0, 0, -np.sqrt(2)/2, 0, 0, np.sqrt(3)/2, 0],
5     [0, 0, 0, -np.sqrt(2)/2, 0, -1, -1/2, 0],
6     [0, 0, 0, 0, -1, 0, 0, 1],
7     [0, 0, 0, 0, 0, 1, 0, 0],
8     [0, 0, 0, 0, 0, 0, -np.sqrt(3)/2, -1],
9     [0, 0, -1, 0, 0, 0, 1/2, 0]
10 ]
11 B_cotkeo = [0, 0, 0, 0, 0, 10000, 0, 0]
12 ten_an_cotkeo = ["F1", "F2", "F3", "f1", "f2", "f3", "f4", "f5"]

```

4.2.3. Khó khăn 1: Phần tử đường chéo bằng 0

Hàng 3 có $a_{33} = 0$ và hàng 5 có $a_{55} = 0$, gây lỗi chia cho 0. Giải pháp: **hoán vị hàng** bằng thuật toán backtracking.

```

1 def tim_hoanvi(A):
2     n = len(A)
3     thu_tu = []
4     da_dung = []
5
6     def thu(cot):
7         if cot == n:
8             return True
9         for hang in range(n):
10            if hang not in da_dung and A[hang][cot] != 0:
11                thu_tu.append(hang)
12                da_dung.append(hang)
13                if thu(cot + 1):
14                    return True
15                thu_tu.pop()
16                da_dung.pop()
17            return False
18
19     if thu(0):
20         return thu_tu
21     print("Khong tim duoc hoan vi phu hop!")
22     return None

```

Giải thích thuật toán:

- Với mỗi cột j (từ 0 đến $n - 1$), thử đặt một hàng chưa dùng sao cho $a_{\text{hàng},j} \neq 0$.
- Nếu không tìm được hàng nào thỏa cho cột hiện tại, quay lui (backtrack) thử phương án khác.
- Kết quả: mảng `thu_tu` chứa chỉ số hàng cần đặt vào từng vị trí.

Sau khi tìm được hoán vị, sắp xếp lại cả A và B :

```

1 def sap_xep_A_B_tenan(A, B, ten_an):
2     thu_tu = tim_hoanvi(A)
3     if thu_tu is None:
4         return None, None, None
5     n = len(B)
6     A_doicho, B_doicho, ten_an_doicho = [], [], []
7     for i in range(n):
8         j = thu_tu[i]
9         A_doicho.append(A[j])
10        B_doicho.append(B[j])
11        ten_an_doicho.append(ten_an[j])
12    return A_doicho, B_doicho, ten_an_doicho

```

Kết quả hoán vị: thứ tự hàng $[0, 1, 7, 2, 4, 5, 3, 6]$, đường chéo sau hoán vị:

$$\text{diag} = [-1, -1, -1, -0.7071, -1, 1, -0.5, -1].$$

Mọi phần tử đường chéo khác 0 $\Rightarrow$ giải quyết vấn đề chia cho 0.

4.2.4. Khó khăn 2: Không hội tụ

Dù đã hoán vị, ma trận vẫn *không chéo trội*. Ví dụ hàng ứng với đường chéo -0.5 :

$$|-0.5| = 0.5 < |-0.7071| + |-1| = 1.7071.$$

Kết quả: cả Jacobi lẫn Gauss–Seidel đều **không hội tụ**.

4.2.5. Giải pháp: Phương trình chuẩn tắc $A^T A x = A^T B$

Như đã chứng minh ở Mục 2.6, ta nhân hai vế với A^T :

```

1 def tinhtoan_cotkeo_binhphuong(N_max, epsilon, A, B):
2     A = np.array(A)
3     B = np.array(B)
4     AT = A.T           # Ma tran chuyen vi A^T
5     ATA = AT @ A       # Ma tran moi: A^T * A
6     ATB = AT @ B       # Ve phai moi: A^T * B
7
8     q_jacobi, x_matrix_jacobi, err_matrix_jacobi = \
9         Jacobi(N_max, epsilon, ATA, ATB)
10    q_gauss_seidel, x_matrix_gauss_seidel, err_matrix_gauss_seidel = \
11        Gauss_Seidel(N_max, epsilon, ATA, ATB)
12
13    # ... lưu file kết quả ...

```

Giải thích từng bước:

1. $AT = A.T$: Tính ma trận chuyển vị A^T (kích thước 8×8).
2. $ATA = AT @ A$: Tích $A^T A$ – ma trận 8×8 , đối xứng, xác định dương.
3. $ATB = AT @ B$: Vector vế phải mới $A^T B$.
4. Áp dụng Jacobi và Gauss–Seidel cho hệ mới $A^T A x = A^T B$.

4.2.6. Minh họa bằng ví dụ nhỏ

Xét hệ 2×2 đơn giản:

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 3 \\ 5 \end{bmatrix}.$$

Đường chéo $a_{11} = 0 \Rightarrow$ Jacobi không áp dụng trực tiếp được. Tính:

$$A^T A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^T \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad A^T B = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^T \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}.$$

Hệ mới $\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix} x = \begin{bmatrix} 5 \\ 8 \end{bmatrix}$ có đường chéo dương, nghiệm $x_1 = 2$, $x_2 = 3$ – trùng với nghiệm hệ gốc.

4.2.7. Kết quả bài toán cột kéo

Bảng 2: Kết quả nghiệm bài toán cột kéo bằng phương trình chuẩn tắc ($\varepsilon = 10^{-9}$).

Ấn số	Jacobi	Gauss–Seidel
F_1	<i>Không hội tụ</i>	−0.000000004
F_2	<i>Không hội tụ</i>	−6339.745962152
F_3	<i>Không hội tụ</i>	−3660.254037842
f_1	<i>Không hội tụ</i>	−8965.754721676
f_2	<i>Không hội tụ</i>	6339.745962149
f_3	<i>Không hội tụ</i>	9999.999999997
f_4	<i>Không hội tụ</i>	−7320.508075684
f_5	<i>Không hội tụ</i>	6339.745962150
Số lần lặp	999 (không hội tụ)	198

Nhận xét:

- Khi áp dụng trực tiếp (kể cả sau hoán vị hàng), cả hai phương pháp đều không hội tụ do ma trận không chéo trội.
- Sau khi biến đổi về phương trình chuẩn tắc $A^T A x = A^T B$, Gauss–Seidel hội tụ thành công; trong khi đó Jacobi vẫn không hội tụ sau 999 bước lặp.
- Nghiệm Gauss–Seidel phù hợp với nghiệm kiểm chứng từ `np.linalg.solve(A, B)`.

4.2.8. Biện luận kết quả sau khi giải

Từ các giá trị trong Bảng 2, có thể kiểm tra và diễn giải nghiệm như sau:

- **Phản lực ngang gần như bằng 0:** $F_1 \approx -4 \times 10^{-9} \text{ N} \approx 0$. Điều này phù hợp với thực tế vì bài toán chỉ có tải trọng ngoài theo phương thẳng đứng, nên tổng lực theo phương ngang của toàn hệ phải xấp xỉ bằng 0.

- **Hai phản lực đứng cân bằng tải ngoài:** $F_2 \approx -6339.75 \text{ N}$ và $F_3 \approx -3660.25 \text{ N}$. Theo quy ước chiều mũi tên đã chọn trong Hình 1, dấu âm cho biết phản lực thực tế ngược chiều giả thiết ban đầu. Hơn nữa,

$$F_2 + F_3 \approx -10000 \text{ N},$$

nên về độ lớn ta có $|F_2| + |F_3| \approx 10^4 \text{ N}$, đúng bằng tải trọng đặt tại nút 3.

- **Một số quan hệ cân bằng được thỏa rất tốt:** từ nghiệm thu được,

$$|f_1| \sin \frac{\pi}{4} \approx 6339.75 \text{ N} \approx |F_2|,$$

và

$$|f_1| \sin \frac{\pi}{4} + |f_4| \sin \frac{\pi}{6} \approx 10000 \text{ N},$$

cho thấy các phương trình cân bằng lực tại các nút được thỏa mãn nhất quán.

- **Ý nghĩa dấu của nội lực:** nếu quy ước ban đầu lấy nội lực dương là *kéo*, thì các thanh có giá trị âm như f_1 và f_4 đang chịu *nép*, còn các thanh có giá trị dương như f_2, f_3, f_5 đang chịu *kéo*. Dấu của nghiệm vì vậy không phải là sai số, mà chứa thông tin cơ học về trạng thái của từng thanh.
- **Về mặt số học:** Gauss–Seidel hội tụ trên hệ chuẩn tắc sau 198 bước lặp, trong khi Jacobi vẫn không hội tụ. Điều này phù hợp với lý thuyết đã trình bày: $A^T A$ là ma trận đối xứng xác định dương nên Gauss–Seidel được đảm bảo hội tụ, còn Jacobi thì không có bảo đảm tương tự.

5. Kết luận

Từ các kết quả tính toán và phân tích, rút ra các kết luận sau:

1. Jacobi và Gauss–Seidel đều là các phương pháp lặp đơn giản, dễ lập trình để giải hệ phương trình tuyến tính.
2. Gauss–Seidel thường hội tụ nhanh hơn Jacobi nhờ sử dụng ngay giá trị mới nhất trong quá trình lặp.
3. Cả hai phương pháp yêu cầu phần tử đường chéo khác 0 và hoạt động tốt nhất khi ma trận chéo trội. Bài 1 minh họa rõ điều này.
4. Khi ma trận không chéo trội (bài toán cột kéo), cần xử lý thêm:
 - **Hoán vị hàng** bằng backtracking để loại bỏ đường chéo 0 – điều kiện cần nhưng chưa đủ.
 - **Phương trình chuẩn tắc** $A^T A x = A^T B$ biến hệ thành dạng đối xứng xác định dương, đảm bảo Gauss–Seidel hội tụ.